JOBSHEET VII

Mata Kuliah: Praktikum Pemrograman Berbasis Objek

Overriding dan Overloading

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom.



Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 2B

D-IV Teknik Informatika

Tanggal: 09 Oktober 2026

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2026-2027

C. Langkah Kerja

Langkah 1: Account Menyediakan Titik untuk Di-override

Code:

```java
public boolean withdraw(double amount) {
    if (!canWithdraw(amount)) {
        return false;
    }
    balance -= amount;
    return true;
}

protected boolean canWithdraw(double amount) {
    return amount > 0 && amount <= balance;
}
```

Hasil:

```
Problems  Output  Debug Console  Terminal

PS C:\Users\IdeaPad\Documents\SEMESTER 3\
\extensions\redhat.java-1.57.0-win32-x64\
\Roaming\Antigravity IDE\User\workspaceSt
 'id.ac.polinema.Main'
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\
```

Penjelasan: Pada langkah ini dilakukan *refactoring* pada kelas Account dengan memisahkan logika validasi penarikan saldo ke dalam method baru protected boolean canWithdraw(double amount). Hak akses protected digunakan agar method ini dapat diakses dan ditulis ulang (*override*) oleh kelas-kelas turunannya (*subclass*) sesuai aturan masing-masing rekening, tanpa perlu mengubah alur utama di dalam method withdraw().

Langkah 2: SavingsAccount Meng-override canWithdraw dan printInfo

Code:

```java
}

@Override
protected boolean canWithdraw(double amount){
    return amount > 0 && (getBalance() - amount) >= MINIMUM_BALANCE;
}

public void printInfo(){
    super.printInfo();
    System.out.println("Account type: Saving, Interest rate: " + interestRate);
}
```

```java
Pidddd, 8 minutes ago | author (Pidddd)
package id.ac.polinema;

import Jobsheet4.id.ac.polinema.Costumer;

Pidddd, 8 minutes ago | author (Pidddd)
public class Main {
    Run | Debug
    public static void main(String[] args) {
        Customer customer = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer, 100000, 0.02);

        boolean result = savings.withdraw(70000);
        System.out.println("Withdraw 70000 allowed :" + result);
        savings.printInfo();
    }      Pidddd, 8 minutes ago • Jobsheet7 - Langkah 1
}
```

Hasil:

```
orkspaceStorage\641953bbf92bcda82cdde5d7
Withdraw 70000 allowed :false
A003 - Rian - balance: 100000.0
Account type: Saving, Interest rate: 0.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3
```

Penjelasan: Kelas SavingsAccount melakukan *overriding* terhadap method canWithdraw() untuk menerapkan aturan khusus tabungan, yaitu sisa saldo setelah penarikan tidak boleh kurang dari batas minimum (MINIMUM_BALANCE sebesar 50.000). Selain itu, method printInfo() juga di-*override* dengan memanggil super.printInfo() terlebih dahulu untuk mencetak informasi dasar dari Account, lalu menambahkan pencetakan informasi tipe akun dan suku bunga (interestRate). Saat diuji menarik 70.000 dari saldo 100.000, penarikan ditolak (false) karena sisa saldo (30.000) melanggar batas minimum.

Langkah 3: CheckingAccount Meng-override canWithdraw dan printInfo

Code:

```java
    }

    @Override
    protected boolean canWithdraw(double amount){
        return amount > 0 && amount <= getBalance() + overdraftLimit;
    }

    @Override
    public void printInfo(){          Pidddd, 15 minutes ago • Jobsheet 7 - Langkah 3
        super.printInfo();
        System.out.println("Account type: Checking, overdraft limit: " + overdraftLimit);
    }
```

```java
Pidddd, 6 minutes ago | author (Pidddd)
package id.ac.polinema;

Pidddd, 6 minutes ago | author (Pidddd)
public class Main {
    Run | Debug
    public static void main(String[] args) {
        Customer customer = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer, 100000, 0.02);
        savings.withdraw(70000);

        Customer customer2 = new Customer("Dewi", "0812-0000-0004");
        CheckingAccount checking = new CheckingAccount("A004", customer2, 100000, 200000);
        boolean result = checking.withdraw(250000);
        System.out.println("Withdraw 250000 allowed?" + result);

        Bank bank = new Bank(10);
        bank.addAccount(savings);
        bank.addAccount(checking);
        bank.printAllAccounts();
    }
}
```

Hasil:

```
orkspaceStorage\641953bbf92bcda82cdde5d7c75e879e\redhat.
Withdraw 250000 allowed?true
A003 - Rian - balance: 100000.0
Account type: Saving, Interest rate: 0.02
A004 - Dewi - balance: -150000.0
Account type: Checking, overdraft limit: 200000.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Progr
lang#3 needs reviewers  ⊗ 0 ⚠ 0  Java: Ready
```

Penjelasan: Kelas CheckingAccount meng-*override* method canWithdraw() agar mengizinkan penarikan melebihi saldo saat ini hingga batas overdraftLimit (amount <= getBalance() + overdraftLimit), serta meng-*override* printInfo() untuk menampilkan informasi batas overdraft. Saat diuji menarik 250.000 dari saldo 100.000 dengan batas overdraft 200.000, penarikan diizinkan (true) sehingga saldo akhir menjadi -150000.0. Pemanggilan bank.printAllAccounts() juga membuktikan bahwa Java secara otomatis menjalankan versi printInfo() milik masing-masing objek subclass.

Langkah 4: Overloading, deposit() dengan Catatan

Code:

```java
    public boolean deposit(double amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }

    public boolean deposit(double amount, String note){
        if (!deposit(amount)) {      You, 1 minute ago • Uncommitted changes
            return false;
        }
        System.out.println(accountNumber + " deposit note: " + note);
        return true;
    }
```

```java
You, 3 minutes ago | authors (Pidddd and one other)
public class Main {        Pidddd, 25 minutes ago • Jobsheet7 - Langkah 1
    Run | Debug
    public static void main(String[] args) {
        Customer customer = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer, 100000, 0.02);

        savings.deposit(50000, "Initial Top Up");
        savings.printInfo();
    }
}

```

Hasil:

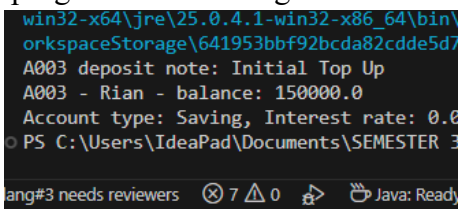
```
win32-x64\jre\25.0.4.1-win32-x86_64\bin\
orkspaceStorage\641953bbf92bcda82cdde5d7
A003 deposit note: Initial Top Up
A003 - Rian - balance: 150000.0
Account type: Saving, Interest rate: 0.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3

ang#3 needs reviewers  ⊗ 7 ⚠ 0  Java: Ready
```

Penjelasan: Langkah ini menerapkan konsep *method overloading* pada kelas Account dengan menambahkan method deposit(double amount, String note) yang memiliki nama sama dengan deposit(double amount), namun berbeda daftar parameternya. Method baru ini memanggil deposit(amount) untuk memvalidasi serta menambahkan saldo, lalu mencetak catatan transaksi (note) jika setoran berhasil, sehingga pengguna memiliki dua alternatif cara untuk melakukan setoran.

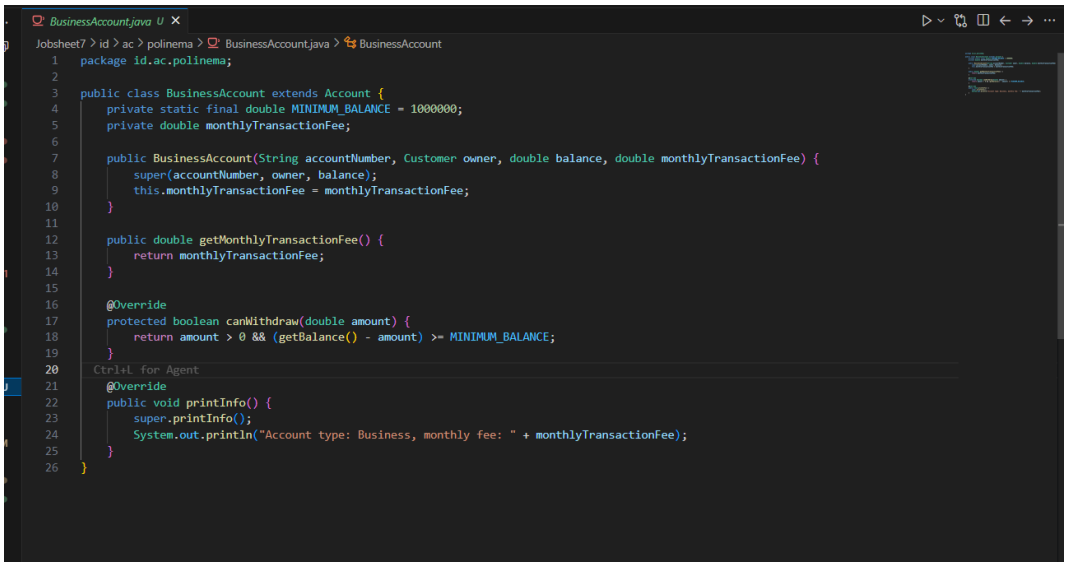
D. Tugas dan Hasil Kerja

- Screenshot output program setelah Langkah 4.



- Tugas mandiri:

1. Terapkan pola overriding yang sama pada BusinessAccount yang kamu buat pada topik Inheritance. Override canWithdraw(double) agar penarikan wajib menyisakan saldo minimum 1000000, dan override printInfo() agar ikut mencetak jenis rekening. Diagram berikut hanya sketsa method yang perlu ditulis ulang, BUKAN kode jadi, isinya diserahkan sepenuhnya padamu:



Buktikan lewat Main.java: buat satu BusinessAccount bersaldo 2000000, coba tarik 1500000, lalu cetak infonya. Hasilmu benar apabila:

- penarikan 1500000 ditolak; 5
- printInfo() mencetak baris milik Account, diikuti baris jenis rekening;
- kedua method memakai @Override.

```java
package id.ac.polinema;      Pidddd, 5 hours ago • Jobsheet7 - Langkah 1

public class Main {

    public static void main(String[] args) {
        Customer customer = new Customer("Budi", "0812-0000-0005");
        BusinessAccount business = new BusinessAccount("A005", customer, 2000000, 25000);

        boolean result = business.withdraw(1500000);
        System.out.println("Withdraw 1500000 allowed? " + result);
        business.printInfo();
    }
}
```

Hasil:

```
Problems 1  Output  Debug Console  Terminal  Ports

PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum
.antigravity-ide\extensions\redhat.java-1.57.0-win
 '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Antigravi
um-Pemrograman-Berbasi-Object-PBO-_2f5b4ee8\bin' '
Withdraw 1500000 allowed? false
A005 - Budi - balance: 2000000.0
Account type: Business, monthly fee: 25000.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum
```

Penjelasan: Kelas BusinessAccount meng-*override* method canWithdraw(double amount) dengan menambahkan aturan bahwa selisih antara saldo saat ini (getBalance()) dan jumlah penarikan (amount) harus lebih besar atau sama dengan saldo minimum 1000000. Selain itu, method printInfo() juga di-*override* menggunakan anotasi @Override dengan memanggil super.printInfo() terlebih dahulu sebelum mencetak jenis rekening bisnis beserta biaya transaksi bulanannya (monthlyTransactionFee). Pada pengujian di Main.java, penarikan sebesar 1500000 dari saldo 2000000 ditolak (false) karena penarikan tersebut hanya akan menyisakan saldo 500000 (di bawah batas minimum 1000000), sehingga saldo akhir tetap utuh sebesar 2000000.0.

2. Jawab secara singkat (1-2 kalimat untuk masing-masing pertanyaan):

   (a) Apa perbedaan signature antara method yang di-override (canWithdraw(), printInfo()) dan method yang di-overload (deposit())?

Jawab: Method yang di-*override* harus memiliki *signature* (nama method serta jumlah, tipe, dan urutan parameter) yang sama persis dengan method di *superclass*-nya. Sebaliknya, method yang di-*overload* memiliki nama method yang sama, tetapi daftar parameternya wajib berbeda (baik dari segi jumlah, tipe data, maupun urutannya).

(b) Siapa yang menentukan versi method yang dijalankan pada overriding, dan siapa pada overloading?

Jawab: Pada *overriding*, versi method yang dijalankan ditentukan oleh JVM pada saat program berjalan (*runtime*) berdasarkan tipe objek aktualnya (*dynamic binding*). Sedangkan pada *overloading*, versi method yang dijalankan ditentukan oleh compiler pada saat kompilasi (*compile-time*) berdasarkan kecocokan argumen yang dikirimkan saat pemanggilan method (*static binding*).

Hasil dan Pembahasan

Program Bank Mini pada praktikum ini telah berhasil mengimplementasikan konsep *Method Overriding* dan *Method Overloading* dalam bahasa pemrograman Java. Melalui *overriding*, ketiga subclass (SavingsAccount, CheckingAccount, dan BusinessAccount) dapat mendefinisikan aturan validasi penarikannya masing-masing pada method canWithdraw() seperti aturan saldo minimum pada rekening tabungan dan bisnis, serta fasilitas *overdraft* pada rekening giro tanpa perlu mengubah logika utama di dalam method withdraw() milik superclass Account. Ketiga subclass juga berhasil meng-*override* method printInfo() dengan memanfaatkan super.printInfo() untuk menampilkan data dasar rekening sekaligus menambahkan informasi spesifik tiap jenis rekening. Selain itu, melalui konsep *overloading*, kelas Account kini menyediakan dua variasi method deposit(), yaitu setoran biasa (deposit(double)) dan setoran yang dilengkapi catatan transaksi (deposit(double, String)), yang dipilih secara otomatis oleh compiler berdasarkan argumen pemanggilannya.

Kesimpulan

Method Overriding memungkinkan subclass mengubah atau memperluas perilaku method yang diwarisi dari superclass dengan mempertahankan *signature* yang sama persis, di mana penentuan method yang dieksekusi dilakukan saat *runtime* oleh JVM berdasarkan objek aktualnya. Sementara itu, *Method Overloading* memungkinkan sebuah kelas memiliki beberapa method bernama sama namun dengan daftar parameter yang berbeda untuk memberikan fleksibilitas cara pemanggilan, di mana pemilihan method dilakukan saat *compile-time* oleh compiler. Kombinasi keduanya membuat struktur kode menjadi jauh lebih fleksibel, modular, dan bebas dari duplikasi logika.

Lampiran

Link Github Ahmad Rafid Riqkullah atau https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet7/id/ac/polinema